VPN and NonVPN Network Traffic Classification Using Machine Learning

Shucayb Ahmed Said, Ayub Ali Barre (Group 17)
Bachelor of Science in Computer Engineering (TGDTA), University West, Sweden
Emails: name1@student.hv.se, name2@student.hv.se (insert student emails before submission)

Abstract

Encrypted Virtual Private Network (VPN) tunnels protect user privacy by hiding packet payloads, which makes traditional payload inspection ineffective for network management and traffic analysis. This project investigates binary classification of VPN and NonVPN network flows using flow-level timing and rate statistics from the ISCXVPN2016 benchmark dataset. From 18,758 raw 15-second flow records, we selected nine numerical features, converted 1,240 invalid negative entries into missing values for training-only median imputation, and removed 725 duplicate and 54 conflicting-label rows, retaining 17,979 clean observations. Using a stratified 80/20 train/test split (14,383 training and 3,596 test flows) and ten-fold stratified cross-validation, we compared three supervised models: DummyClassifier, SGDClassifier, and a regularized Random Forest classifier. In cross-validation, Random Forest achieved a mean validation VPN F1 of 0.8941 and validation accuracy of 88.81%, substantially outperforming DummyClassifier (0.6828 VPN F1) and SGDClassifier (0.6369 VPN F1). On the 3,596-flow reused development-test partition, Random Forest reached 89.29% accuracy, 0.8991 VPN F1 (87.90% precision, 92.01% recall), and 0.9640 ROC-AUC. Although hyperparameter regularization reduced the paired training–validation F1 gap to 0.0466, moderate overfitting remains, motivating future evaluation on newly captured, session-separated network traffic.

***Keywords*—**Artificial Intelligence, Machine Learning, VPN Traffic Classification, Random Forest, Cross-Validation, Overfitting

1. Introduction

A Virtual Private Network (VPN) encrypts network traffic and routes it through an intermediate server to protect user privacy and secure communication. NonVPN traffic is transmitted directly across the network without an encrypted VPN tunnel. Because encryption hides packet payloads and port numbers, network administrators and cybersecurity systems cannot rely on deep packet inspection alone to identify VPN tunnels for quality-of-service management, bandwidth allocation, or security policy enforcement [1].

Machine learning offers a practical way to classify encrypted network traffic by analyzing statistical summaries of packet timing and data rates within each network flow [1], [2]. In this project, we study binary classification of recorded network flows into **NonVPN (0)** and **VPN (1)** using nine numerical flow features from the ISCXVPN2016 benchmark dataset [2]. Following a structured machine learning workflow in Python and Google Colab [4], we clean the raw flow records, perform exploratory data analysis on the training partition, and compare three supervised classifiers—**DummyClassifier**, **SGDClassifier**, and **Random Forest**—using 10-fold stratified cross-validation and a held-out evaluation.

2. Methodology

Dataset Description: We use the 15-second time-window flow dataset (*15s.arff*) from the ISCXVPN2016 benchmark created by the Canadian Institute for Cybersecurity at the University of New Brunswick [1], [2]. To ensure exact reproducibility, the notebook downloads the pinned ARFF file and verifies its SHA-256 checksum (*8b12e8c0...04f7c1563*) [4]. The raw file contains 18,758 flow records and 24 attributes. We retain nine numerical features measuring flow duration, data rates, packet inter-arrival times (IAT), and active/idle intervals: *duration*, *flowPktsPerSecond*, *flowBytesPerSecond*, *min_flowiat*, *max_flowiat*, *mean_flowiat*, *std_flowiat*, *mean_active*, and *mean_idle*. The string label column (*class1*) is mapped to a binary target where labels starting with *VPN-* become **1 (VPN)** and all other labels become **0 (NonVPN)**.

Data Preprocessing: Flow durations, packet rates, and inter-arrival times cannot be physically negative. Inspection of the nine selected features revealed 1,240 negative cells across the raw dataset, which we replaced with *NaN* (missing values). Next, we removed 725 exact duplicate rows (where all nine features and the binary label were identical) and 54 conflicting-label rows (where identical feature vectors appeared with opposite class labels), leaving **17,979 clean observations**. Missing values are imputed using the median of each feature (*SimpleImputer(strategy='median')*) inside a scikit-learn *Pipeline* so that medians are estimated strictly from the training observations in each fold. We note as a methodological limitation that removing the 54 conflicting-label rows prior to splitting used labels from the full source dataset.

Exploratory Data Analysis (EDA): To avoid using information from the test set during data exploration, all EDA summaries are computed on the 14,383-flow training partition. The training classes are well balanced, comprising 6,927 NonVPN flows (48.16%) and 7,456 VPN flows (51.84%). In the raw ISCXFlowMeter output, *duration* is stored in microseconds; we divide by 1,000,000 to express flow duration in seconds. As shown in Fig. 1, most training flows last 15 seconds or less (median = 11.85 s, mean = 10.18 s), while 281 longer flows extend up to 601.41 s, producing strong right skewness. Furthermore, Pearson correlations between individual features and the binary VPN label are close to zero (ranging from -0.0670 for *duration* to +0.0532 for *mean_flowiat*). Weak Pearson values indicate weak linear association rather than absence of predictive signal, motivating non-linear decision trees.

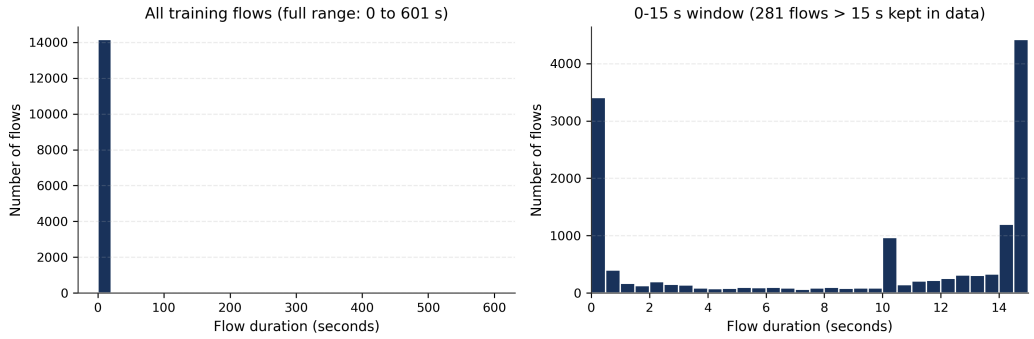


Fig. 1. Distribution of training flow durations in seconds across the full range (left) and within the 0–15 second capture window (right).

Machine Learning Models: We compare three classifiers implemented with scikit-learn Pipelines [3]: (1) **DummyClassifier** is a simple baseline that always predicts the majority class (VPN); we use it to verify whether our machine learning models beat a basic prediction strategy. (2) **SGDClassifier** is a linear machine learning model trained with stochastic gradient descent (default hinge loss and *StandardScaler* standardization); we use it to compare a simple linear boundary with a tree ensemble. (3) **Random Forest** combines predictions from many decision trees to learn non-linear thresholds and interactions between flow features. Rather than running a new hyperparameter search in the final notebook, we use the regularized Random Forest configuration selected during earlier training-only tuning experiments.

3. Experimental Setup

All experiments were executed in Google Colab using *Python 3.13.16*, *scikit-learn 1.6.1* [3], *pandas 2.2.3*, *NumPy 2.1.3*, and *Matplotlib 3.10.7* [4]. The 17,979 cleaned flows were partitioned using a stratified 80/20 split (*train_test_split*, *random_state=42*) into **14,383 training flows** (6,927 NonVPN, 7,456 VPN) and **3,596 test flows** (1,732 NonVPN, 1,864 VPN), with zero identical feature vectors shared across the split. Model comparison used 10-fold stratified cross-validation (*StratifiedKFold(n_splits=10, shuffle=True, random_state=42)*) on the training partition. Our selected **Random Forest** classifier uses *n_estimators=200*, *criterion='entropy'*, *max_depth=14*, *min_samples_leaf=4*, *min_samples_split=10*, *max_features=5*, *max_samples=0.7*, *bootstrap=True*, and *random_state=42*. Because predicting every

flow as VPN yields 100% recall but only 51.84% precision, we use **VPN F1** (positive class *VPN = 1*) as the primary comparison metric alongside accuracy, precision, recall, ROC-AUC, and the paired training–validation F1 gap (mean of fold-training F1 minus fold-validation F1).

4. Results and Discussion

Table 1 summarizes the full-training and 10-fold stratified cross-validation performance across the 14,383 training observations. **DummyClassifier** obtains 51.84% validation accuracy and 0.6828 validation VPN F1 by predicting every flow as VPN (100.00% recall, 51.84% precision). **SGDClassifier** reaches 59.22% validation accuracy, but its mean validation VPN F1 drops to 0.6369 (SD = 0.0326), confirming that a single linear boundary cannot effectively separate encrypted VPN and NonVPN traffic flows. In contrast, **Random Forest** achieves **93.81% training accuracy**, **0.9410 training VPN F1**, **88.81% mean validation accuracy**, and **0.8941 mean validation VPN F1** (SD = 0.0085), with 87.76% validation precision and 91.12% validation recall.

Table 1. Comparison of the three classifiers on the training partition (n = 14,383; 10-fold stratified CV).

Classifier	Train Acc.	Train VPN F1	Val. Acc.	Val. Prec.	Val. Recall	Mean Val. VPN F1 (±SD)	Paired F1 Gap
DummyClassifier	51.84%	0.6828	51.84%	51.84%	100.00%	0.6828 (±0.0003)	0.0000
SGDClassifier	58.93%	0.6420	59.22%	59.07%	69.39%	0.6369 (±0.0326)	-0.0008
Random Forest	93.81%	0.9410	88.81%	87.76%	91.12%	0.8941 (±0.0085)	0.0466

After cross-validation, the selected **Random Forest** pipeline was fitted on all 14,383 training rows and evaluated on the 3,596 held-out test rows (Table 2 lists the exact six-decimal metrics from the notebook output). Because this test partition was previously inspected during earlier project development, it is reported as a *reused development-test evaluation* rather than an untouched independent test. On this partition, Random Forest achieves **89.29% test accuracy**, **0.8991 test VPN F1** (87.90% precision, 92.01% recall), and **0.9640 ROC-AUC**, closely matching its 10-fold validation VPN F1 (0.8941).

Table 2. Selected Random Forest performance across training, 10-fold validation, and reused test evaluation.

Evaluation Stage / Metric	Measured Value	Notes / Partition Size
Training Accuracy / Training VPN F1	93.8052% / 0.940966	Full training partition (n = 14,383 flows)
Mean 10-Fold Validation VPN F1 (±SD)	0.894053 (±0.0085)	10-fold stratified CV on training set (87.76% precision, 91.12% recall)
Paired Training–Validation F1 Gap	0.046553	Mean of (fold train F1 - fold val F1): 0.940606 - 0.894053
Test Accuracy (reused development-test)	89.2937% (3,211 / 3,596)	Reused held-out partition (n = 3,596 flows)
Test VPN Precision / Test VPN Recall	0.879036 / 0.920064	87.90% precision; 92.01% recall (positive class VPN = 1)
Test VPN F1 / Test ROC-AUC	0.899083 / 0.964009	Default class prediction and predicted VPN probabilities

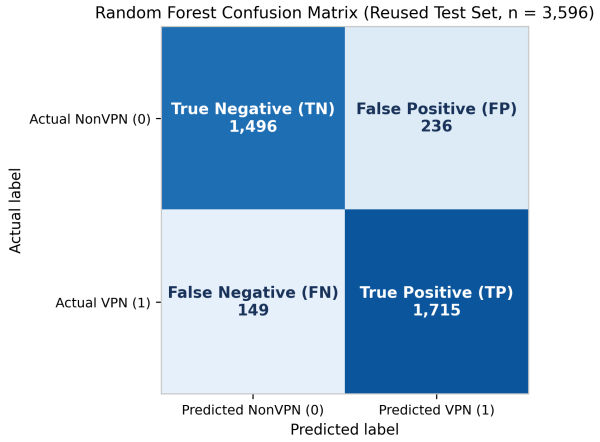


Fig. 2. Random Forest confusion matrix on the reused development-test partition (n = 3,596 flows).

Confusion Matrix, Overfitting, and Runtime Comparison: As shown in Fig. 2 (confusion matrix [[1496, 236], [149, 1715]]), the Random Forest correctly classifies 1,496 NonVPN flows (True Negatives) and 1,715 VPN flows (True Positives), while producing 236 False Positives and 149 False Negatives. In earlier archived tuning experiments [4], an unregularized default Random Forest reached 0.9999 training VPN F1 and 0.9016 validation VPN F1 (paired F1 gap = 0.0984). Restricting tree depth (*max_depth=14*), minimum leaf and split sizes (*4* and *10*), and bootstrap sample size (*max_samples=0.7*) reduced the paired F1 gap by **52.7%** to **0.0466** (mean fold-training F1 of 0.9406 vs. validation F1 of 0.8941), though moderate overfitting remains and regularization traded a small amount of F1 compared with the unregularized forest. Compared with the earlier reference execution (89.3215% test accuracy, 0.899371 test VPN F1, and confusion matrix [[1496, 236], [148, 1716]]), the current Google Colab execution differs on the test set by one borderline VPN observation (flow #1698, whose predicted VPN probability is near the 0.50 threshold), which changed classification from True Positive to False Negative while all NonVPN predictions remained unchanged. Because the dataset checksum, cleaned rows, split indices, and hyperparameters are identical, an environment-related numerical difference is a possible cause for this single-flow shift.

Limitations: Six limitations should be considered when interpreting these findings: (1) a 0.0466 training–validation F1 gap remains; (2) the 20% test partition was inspected during earlier project development and is a reused development-test evaluation; (3) removing 54 conflicting-label rows before splitting relied on full-dataset labels; (4) the ARFF source lacks session or capture identifiers, so flows from the same network session may appear across splits; (5) ISCXVPN2016 represents historical 2016 traffic; and (6) the regularized Random Forest trades slightly lower predictive accuracy for simpler trees and a smaller fit gap.

5. Conclusion

This project implemented a complete, reproducible machine learning workflow for classifying VPN and NonVPN network traffic flows using nine timing and rate features from the ISCXVPN2016 dataset. Across 10-fold stratified cross-validation on 14,383 training flows, **Random Forest** (0.8941 validation VPN F1, 88.81% validation accuracy, 93.81% training accuracy, and 0.9410 training VPN F1) clearly outperformed both the majority-class **DummyClassifier** (0.6828 VPN F1) and the linear **SGDClassifier** (0.6369 VPN F1). On the 3,596-flow reused development-test set, Random Forest achieved 89.29% accuracy, 0.8991 VPN F1 (87.90% precision, 92.01% recall), and 0.9640 ROC-AUC. We learned that tree ensembles capture non-linear traffic timing patterns that linear models miss, and that constraining tree growth reduces (gap = 0.0466) but does not eliminate overfitting. Future work should evaluate this pipeline on newly captured, session-separated network traffic.

Acknowledgment

In accordance with TAM400 course guidelines, the authors acknowledge the use of generative AI tools (ChatGPT, OpenAI Codex, and Google Antigravity/Gemini) to support Python code cleanup, debugging, local execution, visualization formatting, and drafting and editing of the notebook and report explanations. The student authors are responsible for reviewing the code and numerical results, verifying the notebook execution, and understanding and presenting the submitted work.

References

[1] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Encrypted and VPN Traffic Using Time-Related Features," in Proc. 2nd Int. Conf. Information Systems Security and Privacy (ICISSP), Rome, Italy, 2016, pp. 407–414, doi: 10.5220/0005740704070414.

[2] Canadian Institute for Cybersecurity (CIC), "VPN-nonVPN dataset (ISCXVPN2016)," University of New Brunswick. [Online]. Available: https://www.unb.ca/cic/datasets/vpn.html

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org/stable/

[4] S. A. Said and A. A. Barre, "VPN-NonVPN-Classification-project (TAM400 Group 17): Implementation Code, Notebook, and Figures," GitHub repository, 2026. [Online]. Available: https://github.com/SAS2005/VPN-NonVPN-Classification-project